RISC-V и тестовые генераторы

МФТИ
Весна 2026

Кто преподает?

- Тарасов Юлий @botan_razdolb

- Белов Владислав @sick_hoof

Где преподает?

Аудитория – 4.22 (левая) Физтех.Цифра

Слайды – Github [riscv-technologies-lab/testgen-lectures]

Когда преподает?

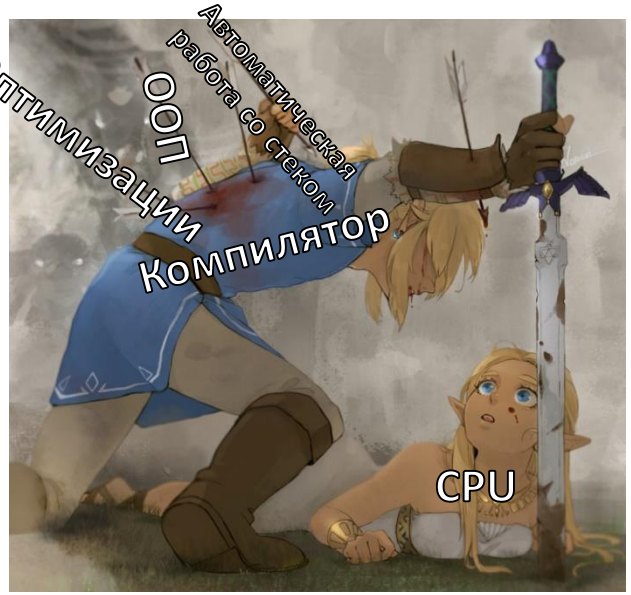
Каждый понедельник 14:00 – 15:20

Что преподает?

- Архитектура и экосистема RISC-V
- Как понять что ваш процессор работает (или нет)
- Методологии тестирования и верификации ПО и аппаратуры
- Функциональный симулятор
- LLVM, MIR
- LLVM-based OS проект llvm-snippy
- Baremetal internals

Уровни абстракции

- Абстракции помогают не сойти с ума
- SW (software) – компилятор и выше
- HW (hardware) – микроархитектура и ниже
- Архитектура – интерфейс между SW и HW



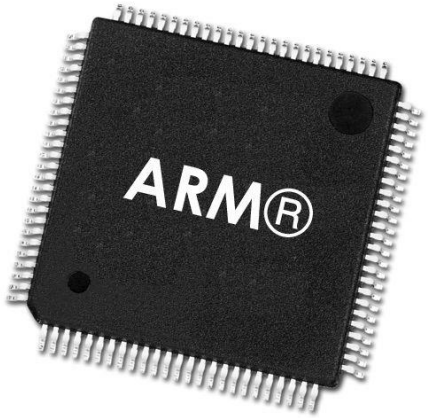
Архитектура определяет …

- Набор команд (как их семантику, так и кодировку)
- Регистры
- Типы данных
- Работу с памятью
- Интерфейсы, ввод/вывод
- Создание прерываний и исключительных состояний и их обработка

Какие архитектуры уже есть

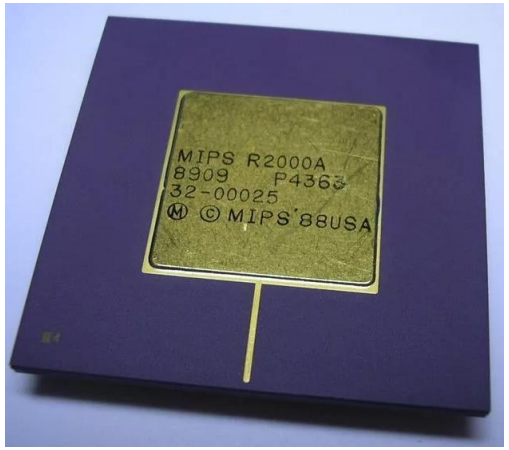
x86
x86-64

Ноутбуки, ПК, сервера



ARM

Мобильные устройства, сервера



MIPS

PSP, сетевые устройства

Эльбрус

ДАННЫЕ УДАЛЕНЫ

Зачем еще одна?

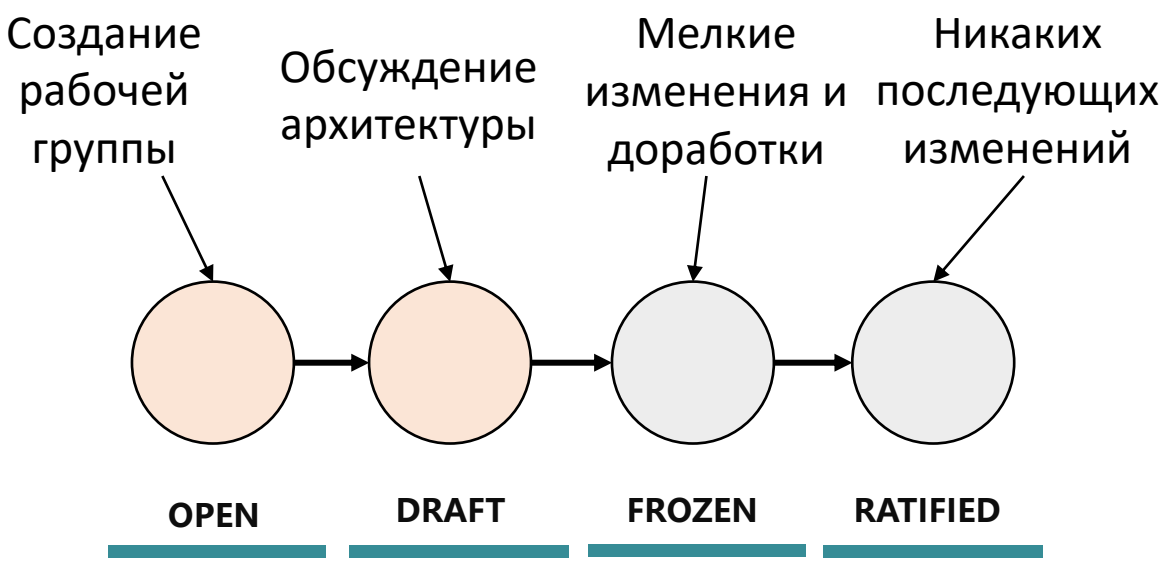
RISC-V

- Открытая
  - Спецификация в открытом доступе, royalty-free
- Свободная
  - Развитие в рамках комитета
- Модульная и расширяемая
  - Минимальный базовый набор
  - Дополнительная функциональность включается через расширения
  - Возможно включать расширения в различных комбинациях

RISC-V Foundation

- Основан в 2015 индустриальными лидерами и стартапами
- 4500+ членов из 70+ стран
- Продвигает исследования и инновации

Процесс ратификации спецификации RISC-V



Альянс RISC-V

Задачи:

- Создание открытого сообщества разработчиков
- Участие в фундаментальных исследованиях
- Развитие российской экосистемы продуктов

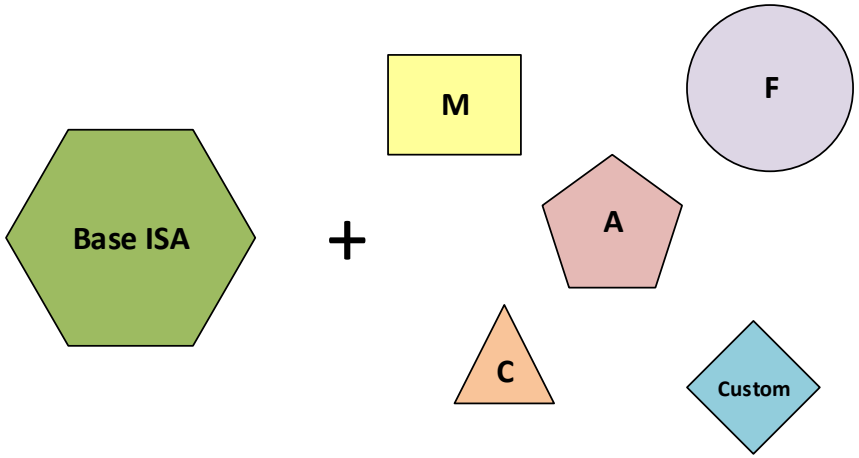
Профильные комитеты:

- Технологический
- Индустриальный
- Юридический
- Академический

Модульность и расширяемость RISC-V

- Базовый набор и стандартные расширения зафиксированы
- Добавление функциональности через расширения, не выпуск новых версий

M	Умножение и деление
C	Сжатые инструкции
F	Single-precision floats
D	Double-precision floats
E	Сокращенное количество регистров
A	Атомики
Z*	Другие стандартные расширения



Минимализм базовой ISA

Jumps & Calls
JAL
JALR
BEQ
BNE
BLT
BGE
BLTU
BGEU

Loads & Stores
LB
LH
LW
LBU
LHU
SB
SH
SW
LWU
LD
SD

Arithmetics			
ADD	ADDI	ADDW	ADDIW
SUB		SUBW	
OR	ORI		
XOR	XORI		
AND	ANDI		
SRL	SRLI	SRLW	SRLIW
SLL	SLLI	SLLW	SLLIW
SRA	SRAI	SRAW	SRAIW
Data flow			
SLT	SLTU	SLTI	SLTIU

Special
FENCE
ECALL
EBREAK

Upper immediate
LUI
AUIPC

Псевдооперации

```
for (i = 0; i < N; ++i)
  if (a[i] == x)
    return i;
```

```
        mv      a5,a0
        li      a0,0
        j       .loop
.latch:
        addiw   a0,a0,1
.loop:
        lw      a4,0(a5)
        addi    a5,a5,4
        bne     a4,a2,.latch
.exit:
        ret
```

Псевдоинструкция	Базовая инструкция	Смысл
nop	addi x0, x0, 0	Нет операции
li rd, immediate	*Различные последовательности*	Загрузка константы
mv rd, rs	addi rd, rs, 0	Копирование регистров
not rd, rs	xori rd, rs, -1	Инверсия числа
neg rd, rs	sub rd, x0, rs	Изменение знака числа
seqz rd, rs	sltiu rd, rs, 1	Установить 1, если == 0
snez rd, rs	sltu rd, x0, rs	Установить 1, если != 0
sltz rd, rs	slt rd, rs, x0	Установить 1, если < 0
sgtz rd, rs	slt rd, x0, rs	Установить 1, если > 0
beqz rs, offset	beq rs, x0, offset	Перейти, если == 0
bnez rs, offset	bne rs, x0, offset	Перейти, если != 0
blez rs, offset	bge x0, rs, offset	Перейти, если <= 0
bgez rs, offset	bge rs, x0, offset	Перейти, если >= 0
bltz rs, offset	blt rs, x0, offset	Перейти, если < 0
bgtz rs, offset	blt x0, rs, offset	Перейти, если > 0
bgt rs1, rs2, offset	blt rs2, rs1, offset	Перейти, если >
ble rs1, rs2, offset	bge rs2, rs1, offset	Перейти, если <=
bgtu rs1, rs2, offset	bltu rs2, rs1, offset	Перейти, если >, беззнаковое
bleu rs1, rs2, offset	bgeu rs2, rs1, offset	Перейти, если <=, беззнаковое
j offset	jal x0, offset	Переход по метке
jal offset	jal x1, offset	Переход с сохранением адреса возврата
jr rs	jalr x0, 0(rs)	Переход по значению из регистра
jalr rs	jalr x1, 0(rs)	Переход с сохранением адреса возврата
ret	jalr x0, x1, 0	Возврат из подпрограммы

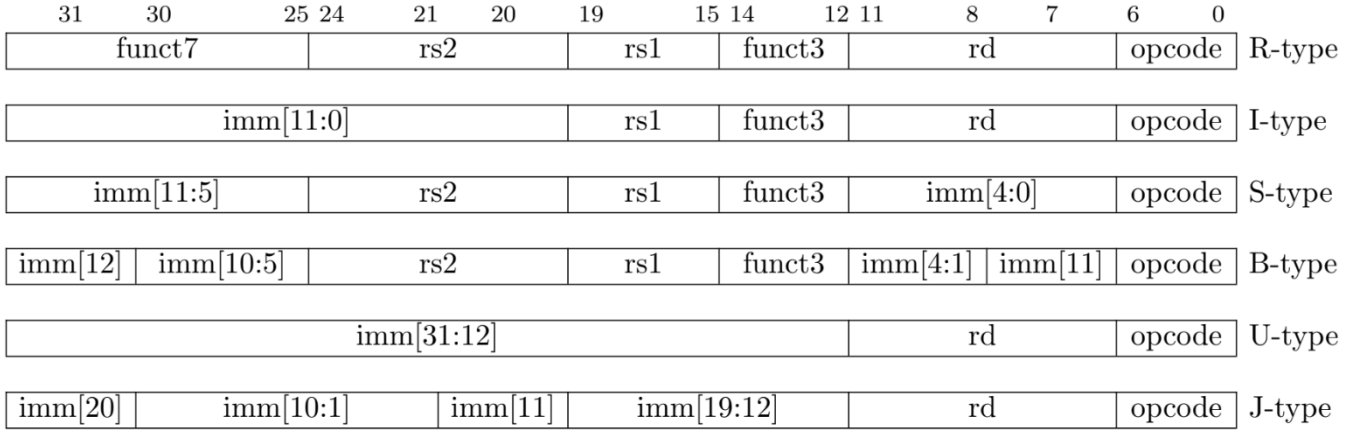
Пример кодировки инструкции

imm [12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				imm [11]	opcode						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	0	0	1	1

Декодер в твоей голове:

- Первые 6 бит – уникальный идентификатор типа инструкции (опкод)
- Каждая инструкция принадлежит какому-то типу
- Знание о типе инструкции даёт всю необходимую информацию для декода инструкции

Формат кодировки



Пример кодировки инструкции

imm [12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				imm [11]	opcode						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	0	0	1	1

Branch

31	30 25	24 21	20 19 15	14 12	11 8	7	6 0	
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type

Пример кодировки инструкции

imm [12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				imm [11]	opcode						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	0	0	1	1

BGE

```
bge ?, ?, ?
```

31	30 – 25	24 – 21	20 – 19	15 – 14	12 – 11	8 – 7	6 – 0	
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type

Пример кодировки инструкции

imm [12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				imm [11]	opcode						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	0	0	1	1

rs2 = x10

rs1 = x0

```
bge x0, x10, ?
```

31	30 25	24 21	20 19 15	14 12	11 8	7	6 0	
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type

Пример кодировки инструкции

imm [12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				imm [11]	opcode						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	0	0	1	1

1	1	1	1	1	0	0	0	0	1	0	0
12	11	10	9	8	7	6	5	4	3	2	1

-248 → `bge x0, x10, -248`

31	30 25	24 21	20 19 15	14 12	11 8	7	6 0	
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode	B-type

Пример кодировки инструкции

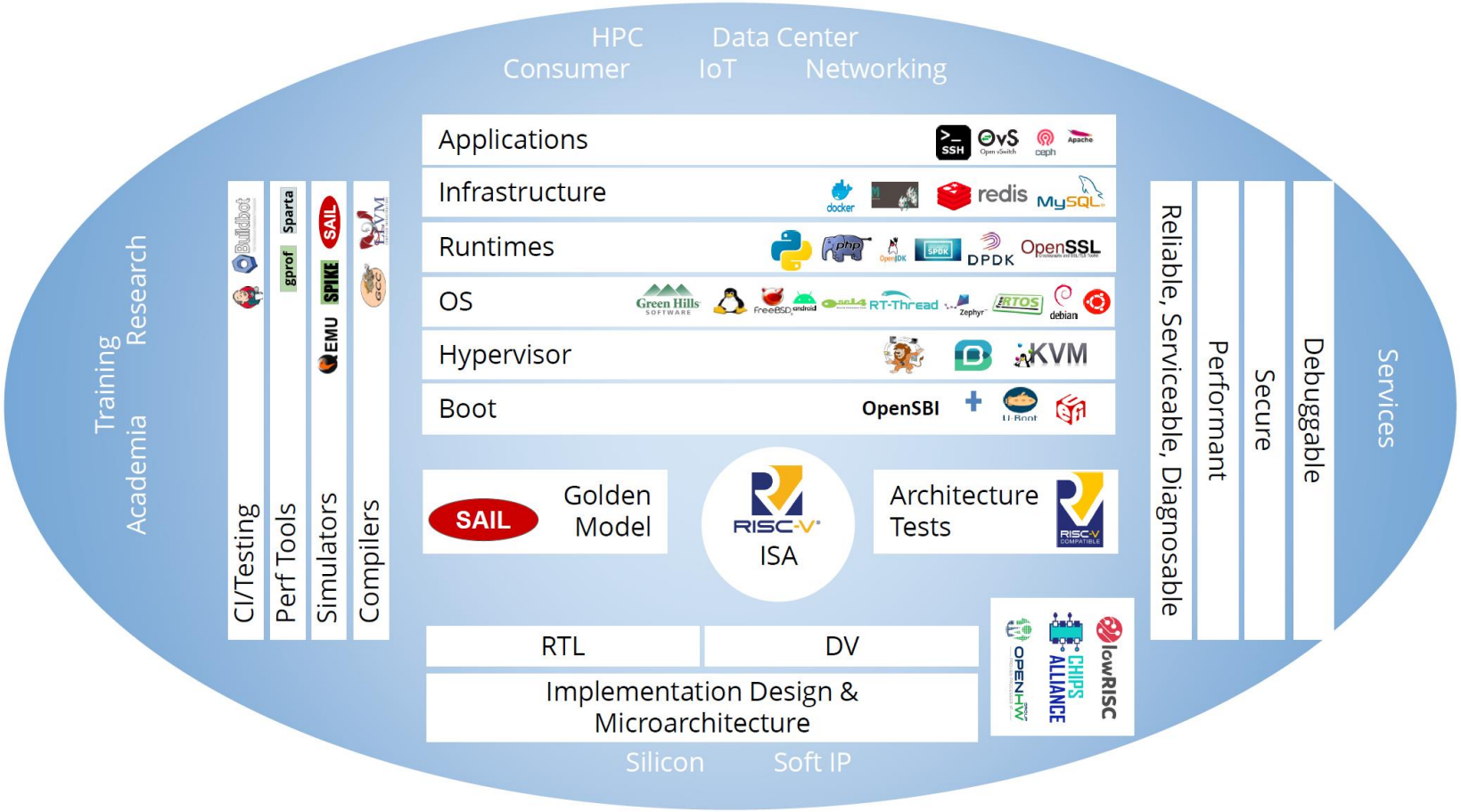
imm [12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				imm [11]	opcode						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	1	0	0	1	1	1	0	0	0	1	1

```
bge x0, x10, -248
```

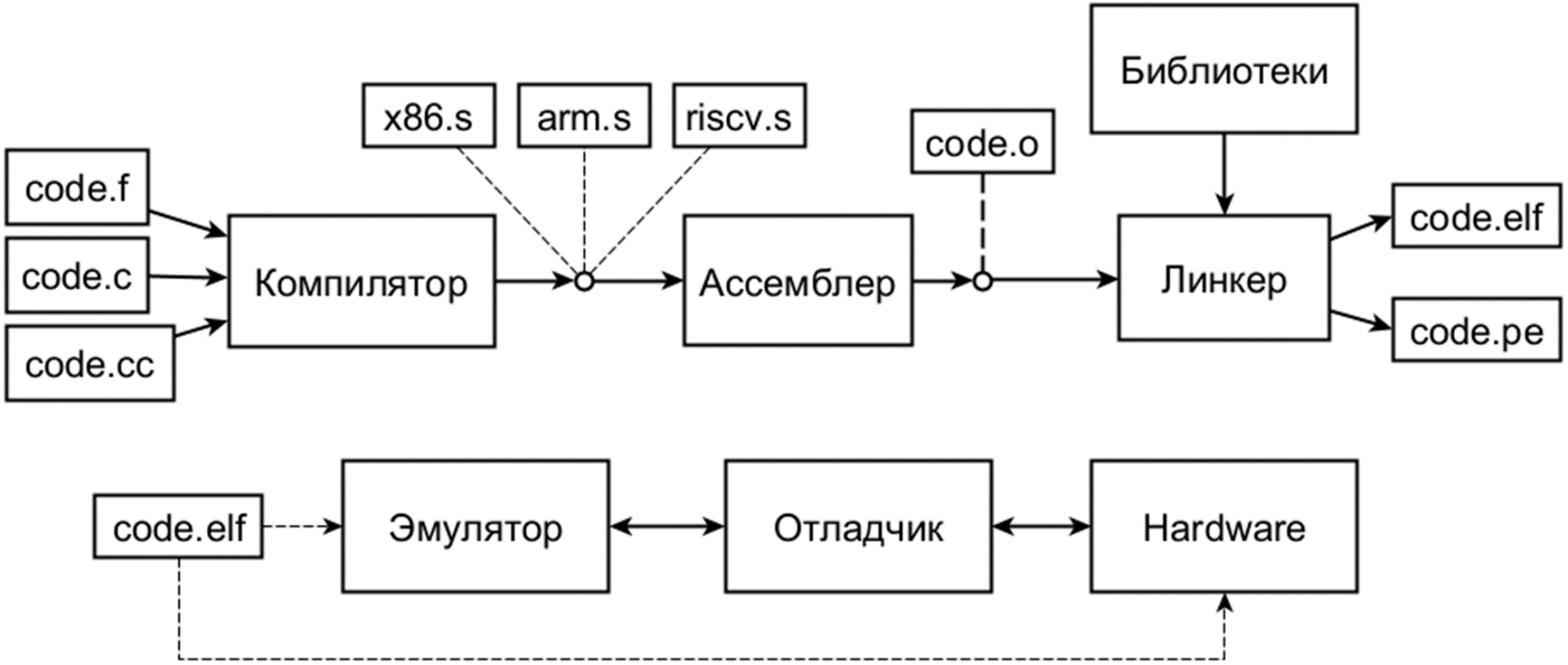
Обсуждение:

- Знаете ли вы ещё какие-нибудь кодировки?
- Подумайте о плюсах и минусах такого формата кодировки
- Как бы вы написали программу, которая декодировала бы инструкции RISC-V ISA?

Другая сторона айсберга



Тулчейн – общий вид



Кросс-компиляция

Основная терминология:

- **build** – машина, где происходит сборка компилятора.
- **host** – машина, где происходит сборка приложения.
- **target** – машина, где происходит запуск приложения.

Обычно для RISCV: build = host, host ≠ target

```
$ riscv64-linux-gnu-gcc -march=rv64 file.c -O2 –S
```

Если host = target, то это нативная компиляция

Имя компилятора на хосте включает в себя target triple

```
<arch><sub>-<vendor>-<sys>-<env>
```

В рассматриваемом случае:

```
arch = riscv, sub = 64, vendor empty, sys = linux, env = gnu
```

Hello, RISC-V!

```
$ cat hello.c
#include <stdio.h>
int main() { printf("Hello, world!\n"); }

$ uname -m
x86_64

$ riscv64-linux-gnu-gcc hello.c -static -o hello.x

$ qemu-riscv64 hello.x
Hello, world!
```

Hello, RISC-V!

```
$ cat hello.c
#include <stdio.h>
int main() { printf("Hello, world!\n"); }

$ uname -m
x86_64

$ riscv64-linux-gnu-gcc hello.c -static -o hello.x

$ qemu-riscv64 hello.x
Hello, world!
```

Всегда ли нужна плата?

- `qemu-riscv32` и `qemu-riscv64`, из sc-dt позволяют запускать RISC-V приложения
- В Syntacore Development Toolkit так же есть `qemu-system-32` и `qemu-system-64`, которые позволяют симулировать систему целиком и фактически являются виртуальной машиной
- Например, в system qemu можно загрузить линукс и протестировать кросс отладку, пока плата еще не готова

Концепция сжатых инструкций

- 4 байта на одну инструкцию это довольно много
- Большой размер одной инструкции
  - Создает излишнюю нагрузку на кэш инструкций
  - Увеличивает размер программ (критично для встраиваемых систем)
- Для решения этих проблем было создано С-расширение, которое добавляет «сжатые» (compressed) инструкции

Концепция сжатых инструкций

- Из спецификации:
  - Typically, 50%-60% of the RISC-V instructions in a program can be replaced with RVC instructions, resulting in a 25%-30% code-size reduction.
- RVC uses a simple compression scheme that offers shorter 16-bit versions of common 32-bit RISC-V instructions when:
  - the immediate or address offset is small, or
  - one of the registers is the zero register (x0), the ABI link register (x1), or the ABI stack pointer (x2), or
  - the destination register and the first source register are identical, or
  - the registers used are the 8 most popular ones.

Концепция сжатых инструкций

```
// -march=rv64i
0000000000000000 <elt>:
   0:   00259593 slli    a1, a1, 0x2
   4:   00a58533 add     a0, a1, a0
   8:   00852503 lw      a0, 8(a0)
```

```
// -march=rv64ic
0000000000000000 <elt>:
   0:   058a    c.slli   a1, a1, 0x2
   2:   952e    c.add    a0, a0, a1
   4:   4508    c.lw     a0, 8(a0)
```

Собираем с расширениями

```
$ riscv64-linux-gnu-gcc file.c –march=rv64i
–mabi=lp64 –static –O2 –o app
$ riscv64-linux-gnu-gcc file.c –march=rv64im
–mabi=lp64 –static –O2 –o app
$ riscv64-linux-gnu-gcc file.c –march=rv64imf
–mabi=lp64f -static –O2 –o app
$ riscv64-linux-gnu-gcc file.c –march=rv64imfd
–mabi=lp64d -static –O2 –o app
$ riscv64-linux-gnu-gcc file.c –march=rv64gc
–mabi=lp64d -static –O2 –o app
```

Что находится на эмблеме факультета?

Рождение новой архитектуры

Рождение новой архитектуры

Рождение новой архитектуры

Рождение новой архитектуры

Как разрабатывают аппаратуру?

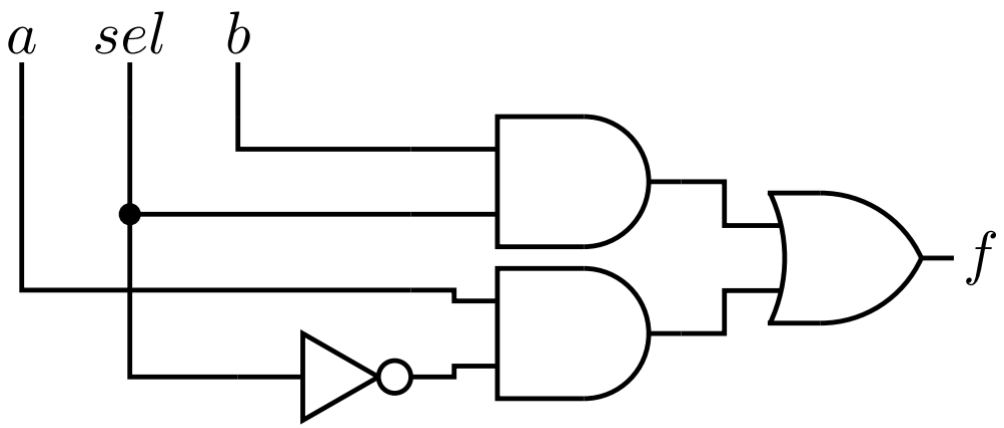
Микроархитектуру разрабатывают на языке описания аппаратуры (**HDL** – ***Hardware Description Language***)

Описание микроархитектуры так же часто называют **RTL** (***Register-Transfer Level***)

Из **RTL** *синтезируют* цифровую схему

Пример RTL модуля на SystemVerilog

```
module mux (
     input  logic a, b, sel,
     output logic f
);
     logic n_sel, f1, f2;
     assign sel = ~n_sel;
     assign f1 = a & n_sel;
     assign f2 = b & sel;
     assign f = f1 | f2;
endmodule
```



RTL готов – отправляем на tapeout

1. Синтезирование из RTL цифровой схемы (на уровне транзисторов) с использованием стандартной библиотеки под конкретную фабрику
2. Проектирование физического расположения проводов и базовых элементов
3. Изготовление фотомаски (tapeout)

Стандартная библиотека – набор базовых элементов, оптимизированных на физическом уровне

С youtube канала Intel

Первый инженерный образец

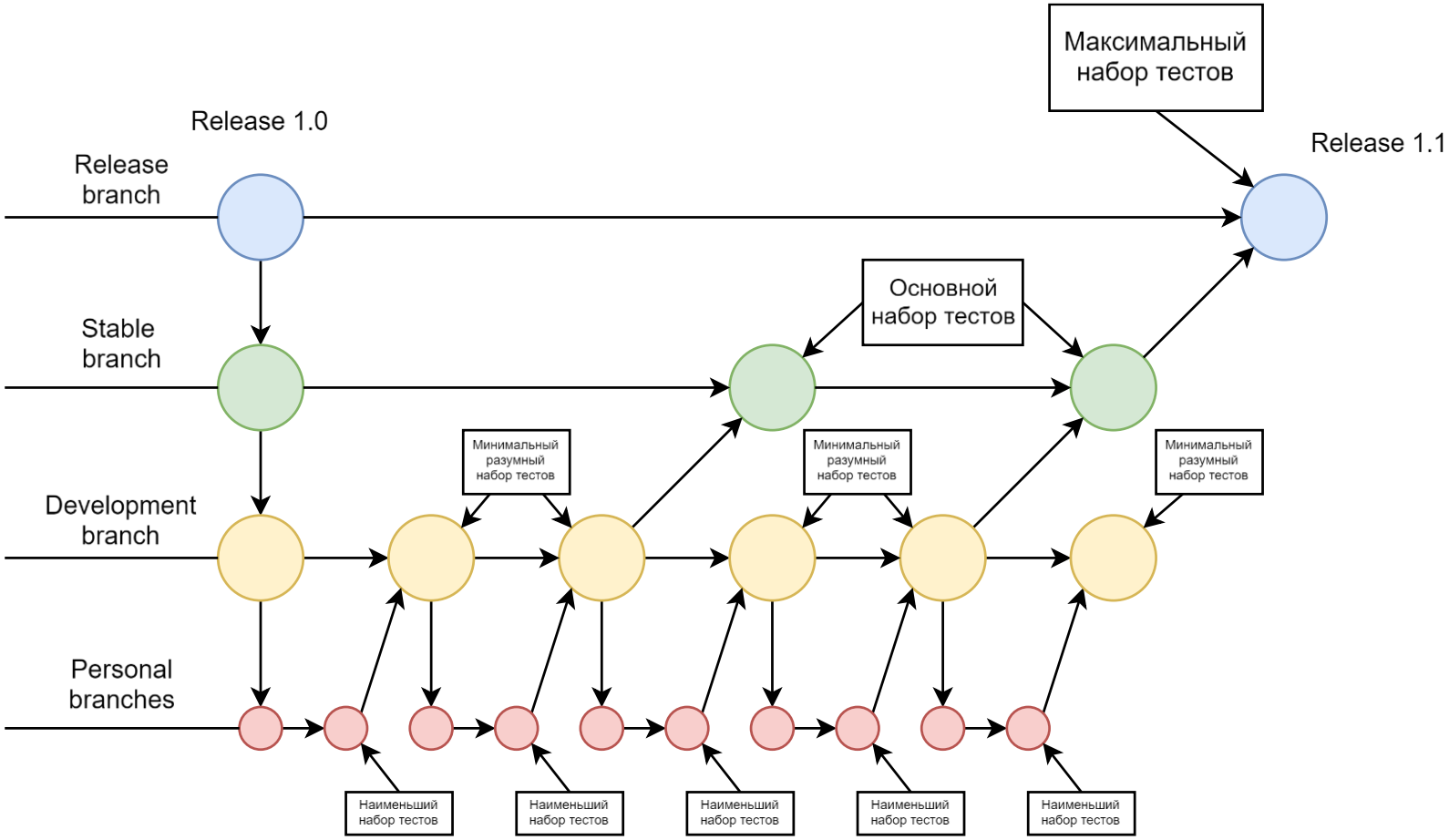
Ура, отгружаем заказчикам!

Или не все так радужно?...

А что вообще у нас получилось?

Поэтапное тестирование

- Sanity
- Дымовое (smoke)
- Регрессионное
- Приемное (релизное)



Обсуждение: как проверить результат теста?

- Сколько тестов вы сможете проверить вручную за час?

Обсуждение: как проверить результат теста?

- Сколько тестов вы сможете проверить вручную за час?

- Средний запуск тестирования включает миллионы тестов
- Оцените сколько будет стоить верификация одного запуска тестов, если один час работы верификатора стоит 12$

- Предложите как можно упростить проверку тестов?

Обсуждение: упрощение проверки тестов

- Основной подход в автоматизированной проверке результатов теста – Model Checking
- Model Checking подразумевает наличие референсной модели (поведение модели считается эталонным)
- При данном подходе с помощью набора тестов происходит сравнение поведения тестируемого объекта с моделью
- В случае тестирования микропроцессора в качестве модели выступают различные симуляторы

Модель или симулятор?

В рамках данного курса:

Модель:

- Моделирует конкретные аспекты системы
- Надо настраивать и конфигурировать вручную

Симулятор:

- Имитирует работу вычислительной системы целиком
- Обеспечивает окружение для выполнения

To be continued …

На следующем занятии узнаем

- Какими бывают разные симуляторы
- В каких сценариях применяются разные симуляторы
- Как отличается производительность разных симуляторов
- Как проверить что вы написали корректный симулятор

Список литературы

- The RISC-V Instruction Set Manual Volume I Unprivileged Architecture Version 20240411 // Chapter 2. RV32I Base Integer Instruction Set, Version 2.1
- Лекция на неофициальном открытии лаборатории RISC-V технологий в МФТИ, февраль 2024: https://youtu.be/xY_Ne9ZznJ4
- Hennessy J. L., Patterson D. A. Computer architecture: a quantitative approach. – Morgan kaufmann, 2017.
- Инструменты программирования для открытой архитектуры RISC V. Константин Владимиров, конференция True Tech, 2024: https://youtu.be/qoNjayusCX4
- Расширяемая архитектура RISC-V и Syntacore SW Tools. Константин Владимиров, конференция «Салют, OS DevConf!», 2024: https://youtu.be/1zLxxxLc0xI
- Харрис Д. М., Харрис Д. Цифровая схемотехника и архитектура компьютера RISC-V. – 2022.